

Data-Driven Optimization

Google Ortools

홍익대학교 산업데이터공학과

옥창수

(okcs@hongik.ac.kr)

데이터 기반 최적화

- "데이터 기반 최적화는 단순히 과거를 분석하는 것을 넘어, 데이터 분석(Analytics)을 의사결정(Decision Making)의 직접적인 입력값으로 전환하는 과정이다. 이는 '무엇이 일어날 것인가(Prediction)'에 대한 답을 '무엇을 해야 하는가(Prescription)'로 연결하는 다리 역할을 한다." - **Dimitris Bertsimas** (MIT 교수, 현대 최적화의 대부) -

Bertsimas, D., & Kallus, N. (2020). "From predictive to prescriptive analytics." Management Science.

- "데이터는 완벽하지 않으며 항상 노이즈와 변동성을 동반한다. 진정한 의미의 데이터 기반 최적화는 데이터의 확률적 분포와 불확실성을 모델의 제약 조건으로 수용하여, 최악의 시나리오에서도 강건(Robust)하게 작동하는 '살아있는 해'를 찾는 학문이다. " - **Aharon Ben-Tal & Arkadi Nemirovski** (강건 최적화의 창시자들) -

Ben-Tal, A., El Ghaoui, L., & Nemirovski, A. (2009). "Robust Optimization." Princeton University Press.

- "최적화 모델에서 파라미터(Parameter)는 더 이상 고정된 상수가 아니라, 데이터로부터 학습된 동적인 신호이다. 데이터 기반 최적화의 목적은 수리적 완결성이 아니라, 데이터에 숨겨진 패턴을 포착하여 실질적인 비즈니스 비용을 최소화하고 운영 효율을 극대화하는 데 있다." - **Prescriptive Analytics** (처방적 분석) -

Lustig, I. J., et al. (2010). "The analytics journey." Analytics Magazine. (IBM 연구팀의 정의)

Predict-then-Optimize (PTO) 프레임워크

- **Data Layer:** 외부 환경(날씨, 유가, 수요) 데이터 수집.
 - **Prediction Layer:** ML 모델을 통한 미래 지표 예측.
 - **Optimization Layer:** 예측값을 입력으로 한 OR-Tools 모델 구동.
- 예측 오차가 최종 의사결정 비용에 미치는 영향(Regret)을 최소화하는 것이 목표.

불확실성 관리: "예측은 항상 틀린다"

- 단일 예측값(Point Estimate)의 위험성 경고.
 - **Stochastic:** 시나리오별 확률 분포를 모델에 반영.
 - **Robust:** 최악의 상황(Worst-case)에서도 작동하는 '안전 마진' 확보.

Deterministic

Real-world Impact: 산업별 혁신 사례

- **VPP(가상발전소):** 기상 예측 기반 에너지 믹스 최적화.
 - **Smart Logistics:** 실시간 주문 데이터를 반영한 동적 차량 경로 관리.
 - **Finance:** 시장 변동성 데이터를 활용한 강건한 포트폴리오 구축.
- "Optimizing the Future, Driven by Data."

통계/최적화/프로그램

데이터 기반 최적화

- **Optimization problems** are mathematical models that seek to find the best solution among a set of feasible alternatives, according to some objective function and constraints.
- **Data-driven optimization** is a branch of optimization that uses data as a primary source of information and knowledge to formulate and solve optimization problems.
- **Data-driven optimization** can leverage data from various sources, such as sensors, databases, surveys, and simulations, to improve the accuracy, reliability, and robustness of optimization models and solutions.
- **Robust Optimization**: 파라미터의 불확실성을 고려한 최적화

강의내용

주차	주제	주요 내용 및 학습 포인트
1	강의 개요 및 OR-Tools 기초	데이터 기반 최적화 개요 및 orootls 사용법
2	선형 및 정수 계획법 복습	수리 모델링(Formulation) 기초, 목적함수와 제약식 설정
3	최적화 모형 파라미터 추출	Raw Data로 부터 파라미터(Cost, Demand)를 결정하는 방법
4	선형 계획법과 민감도 분석	데이터 변화에 따른 최적해의 안정성 및 Shadow Price
5	ML과 정수 계획법 (1): 군집화	Unsupervised Learning(K-means) 결과물을 활용한 자원 할당 및 입지 최적화
6	ML과 정수 계획법 (2): 논리 변환	의사결정나무(Decision Tree)의 규칙을 정수 제약식(If-Then)으로 모델링
7	중간고사	
8	Predict-then-Optimize 실습	수요 예측값($\hat{y}$)을 이용한 최적화 실습
9	Simulation 기반 최적화 (1)	확률 분포 기반 시나리오 생성 및 무작위성(Randomness) 데이터 이해
10	Simulation 기반 최적화 (2)	시뮬레이션 기반 최적화(Simulation-Optimization)를 통한 기댓값 극대화
11	Stochastic Programming	불확실한 미래 시나리오별 대응(Recourse) 전략을 포함한 최적화
12	Robust Optimization	데이터 오차의 최악의 상황(Worst-case)에서도 유효한 강건 모델링 설계
13	산업별 사례연구 (Case Study)	에너지(VPP), 물류, 마케팅 믹스 등 실제 데이터 기반 최적화 사례
14	산업별 사례연구 (Case Study)2	에너지(VPP), 물류, 마케팅 믹스 등 실제 데이터 기반 최적화 사례2
15	기말고사	

수강생들의 이해도에 따라 강의 내용 일부 변경 할 수 있음.

중간고사 및 기말고사

- 중간고사
 - 장소: 추후공지
 - 시간: 4월 23일 (목) 13:00 – 15:00 (120분간)
- 기말고사
 - 장소: 추후공지
 - 시간: 6월 11일 (목) 13:00 – 15:00 (120분간)

1. OR-Tools

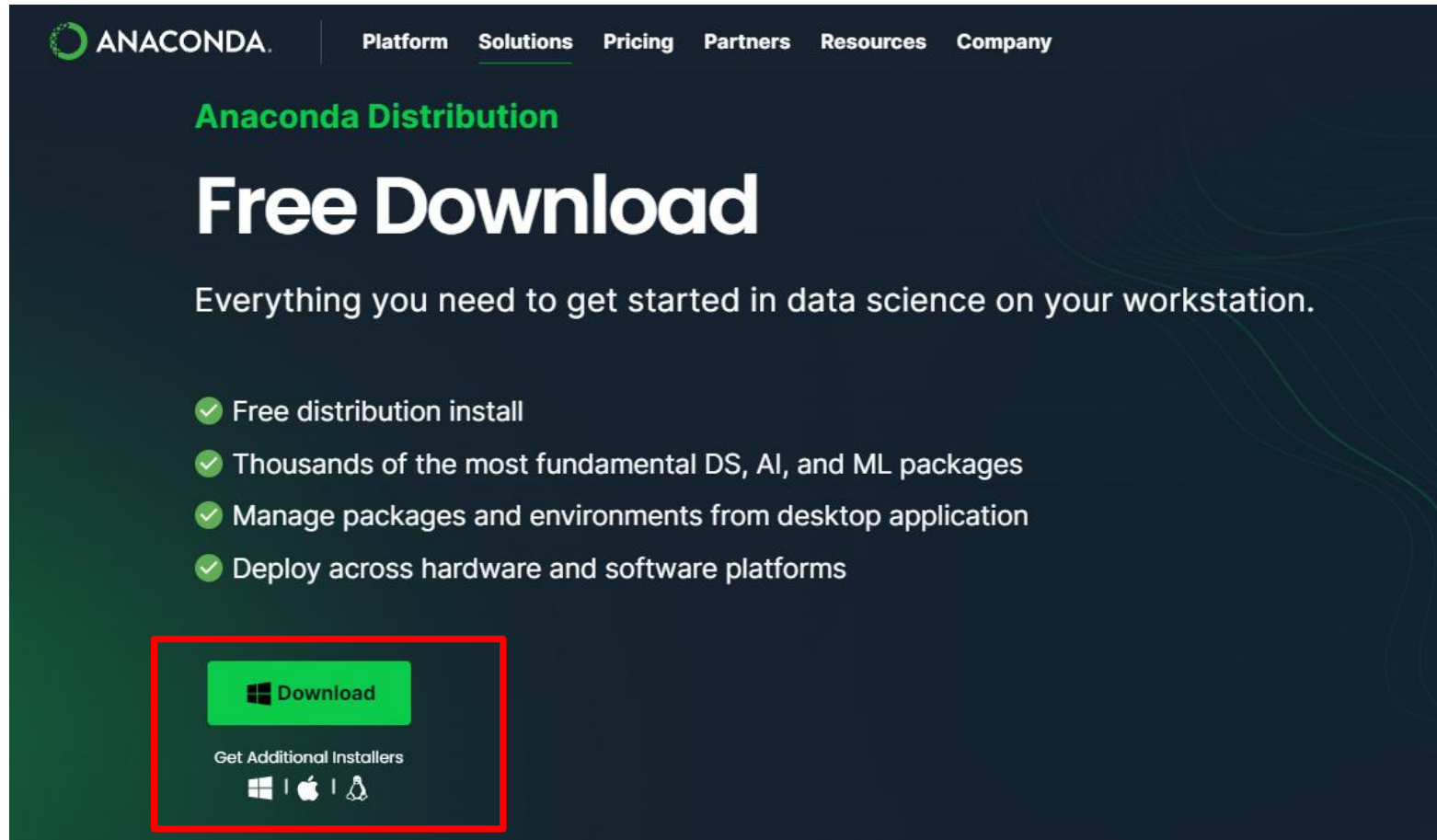
- OR-Tools는 최적화를 위한 오픈소스 소프트웨어 제품군으로, 선형문제, 차량 라우팅, 흐름, 정수 및 선형 프로그래밍, 제약 조건 프로그래밍 등 세계에서 가장 까다로운 문제를 처리하는 데 최적화되어 있음.
- 원하는 프로그래밍 언어로 문제를 모델링한 후 Gurobi 또는 CPLEX와 같은 상용 문제 해결사나 SCIP, GLPK 또는 Google의 GLOP와 같은 오픈소스 솔버와 수상 경력에 빛나는 CP-SAT 등 6개의 솔버 중 하나를 사용하여 문제를 해결할 수 있음.

1. Python용 OR-Tools 설치

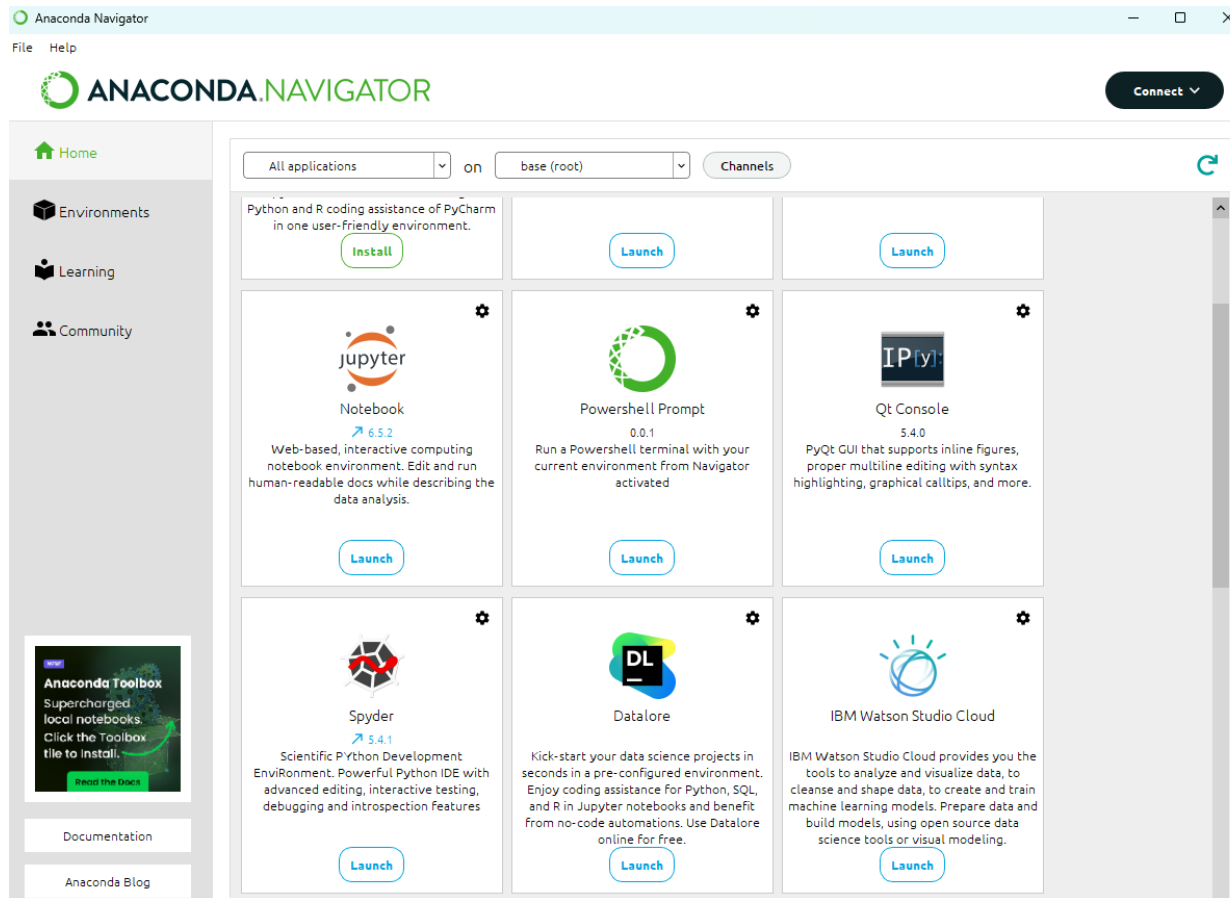
```
python -m pip install --upgrade --user ortools
```

2. Anaconda 설치

ANACONDA 설치 URL : <https://www.anaconda.com/download>



3. Anaconda Navigator에서 Spyder 실행

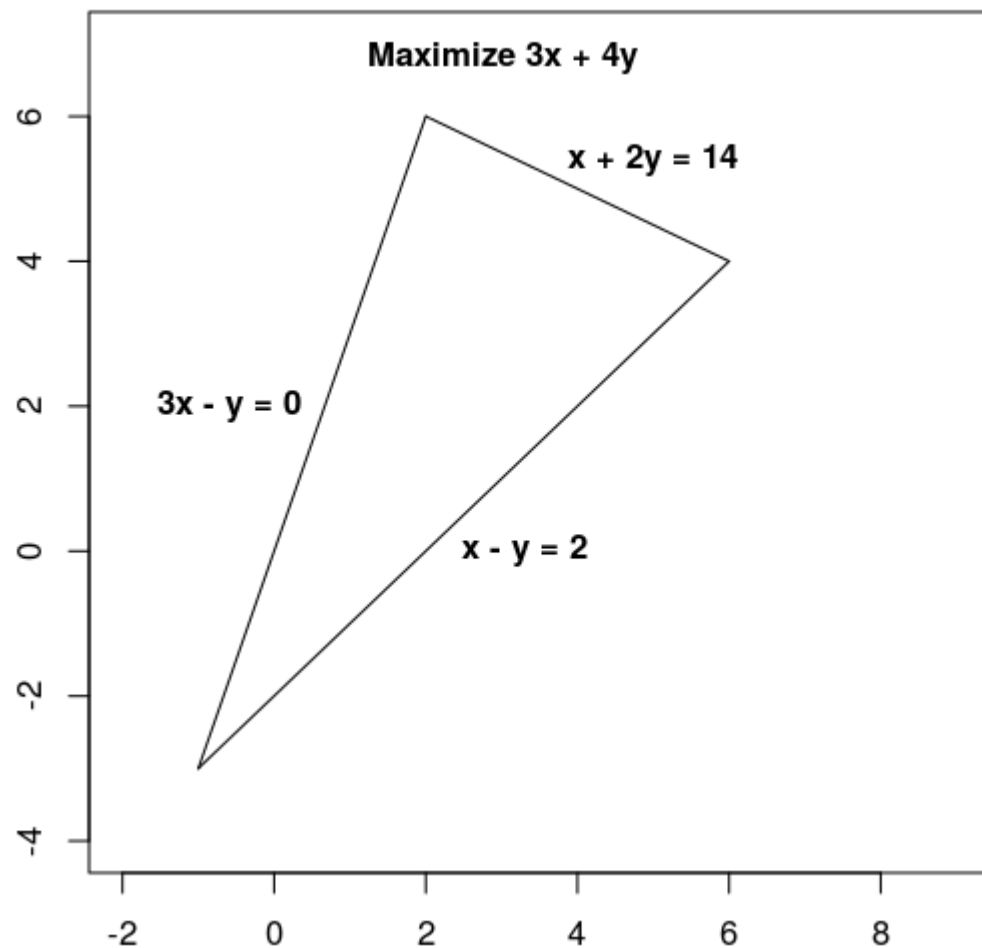


테스트 예제 문제

다음 제약 조건에 따라 $3x + 4y$ 를 최대화합니다.

1. $x + 2y \leq 14$
2. $3x - y \geq 0$
3. $x - y \leq 2$

목표 함수 $3x + 4y$ 와 제약 조건은 모두 선형 표현식으로 주어지므로 선형 문제가 됩니다.



Spyder Editor

Editor

The screenshot displays the Spyder Editor interface. The main window is divided into three panes. The left pane shows a file explorer with a tree view of the project files. The middle pane is the code editor, displaying a Python script named 'orl-0.py'. The script defines a function 'LinearProgrammingExample()' that uses the 'ortools' library to solve a linear programming problem. The script includes comments for variables, constraints, and the objective function. The right pane is the IPython console, showing the output of the script execution. The console output indicates that the problem was solved successfully, providing the optimal solution values for x and y, and the objective function value.

```
1 """Linear programming example."""
2 from ortools.linear_solver import pywraplp
3
4 def LinearProgrammingExample():
5     """Linear programming sample."""
6     solver = pywraplp.Solver.CreateSolver("GLOP")
7     if not solver:
8         return
9     # [END solver]
10
11     # [START variables]
12     x = solver.NumVar(0, solver.infinity(), "x")
13     y = solver.NumVar(0, solver.infinity(), "y")
14     print("Number of variables =", solver.NumVariables())
15     # [END variables]
16
17     # [START constraints]
18     # Constraint 0: x + 2y <= 14.
19     solver.Add(x + 2 * y <= 14.0)
20     # Constraint 1: 3x - y >= 0.
21     solver.Add(3 * x - y >= 0.0)
22     # Constraint 2: x - y <= 2.
23     solver.Add(x - y <= 2.0)
24
25     print("Number of constraints =", solver.NumConstraints())
26     # [END constraints]
27
28     # [START objective]
29     # Objective function: 3x + 4y.
30     solver.Maximize(3 * x + 4 * y)
31
32     # Solve the system.
33     # [START solve]
34     print(f"Solving with {solver.SolverVersion()}")
35     status = solver.Solve()
36     # [END solve]
37
38     # [START print_solution]
39     if status == pywraplp.Solver.OPTIMAL:
40         print("Solution:")
41         print(f"Objective value = {solver.Objective().Value():0.1f}")
42         print(f"x = {x.solution_value():0.1f}")
43         print(f"y = {y.solution_value():0.1f}")
44     else:
45         print("The problem does not have an optimal solution.")
46     # [END print_solution]
47
48     # [START advanced]
49     print("\nAdvanced usage:")
50     print(f"Problem solved in {solver.wall_time():d} milliseconds")
51     print(f"Problem solved in {solver.iterations():d} iterations")
52     # [END advanced]
53
54 LinearProgrammingExample()
```

Python 3.10.16 | packaged by Anaconda, Inc. | (main, Dec 11 2024, 1929 64 bit (AMD64))
Type "copyright", "credits" or "license" for more information.

IPython 8.30.0 -- An enhanced Interactive Python. Type '?' for help

In [1]: %runfile 'C:/Users/1007/Dropbox/04_광의/2025-1/00_데이터기반최적화/01_python_예제파일/orl-0.py' --wdir
Number of variables = 2
Number of constraints = 3
Solving with Glop solver v9.11.4210
Solution:
Objective value = 34.0
x = 6.0
y = 4.0

Advanced usage:
Problem solved in 1 milliseconds
Problem solved in 2 iterations

In [2]:

결과창

최적화 모형 코딩

```
"""Linear optimization example."""
from ortools.linear_solver import pywraplp

def LinearProgrammingExample():
    """Linear programming sample."""
    solver = pywraplp.Solver.CreateSolver("SCIP")
    if not solver:
        return

    # variables
    x = solver.NumVar(0, solver.infinity(), "x")
    y = solver.NumVar(0, solver.infinity(), "y")
    print("Number of variables =", solver.NumVariables())

    # constraints
    solver.Add(x + 2 * y <= 14.0)
    solver.Add(3 * x - y >= 0.0)
    solver.Add(x - y <= 2.0)
    print("Number of constraints =", solver.NumConstraints())
```

pywraplp는 기본 C++ 솔버를 위한 Python Wrapper.

인수 "SCIP"는 "GLOP", "SAT", "GLPK", Gurobi, "CPLEX"등을 사용할 수 있음.
최적화 모형 solver를 지정하는 것으로 필요에 따라 다양한 엔진을 사용할 수 있음.

의사결정변수

제약조건

최적화 모형 코딩 (2)

```
# objective function
solver.Maximize(3 * x + 4 * y)
```

목적함수

```
# Solve the system.
print(f"Solving with {solver.SolverVersion()}")
status = solver.Solve()
```

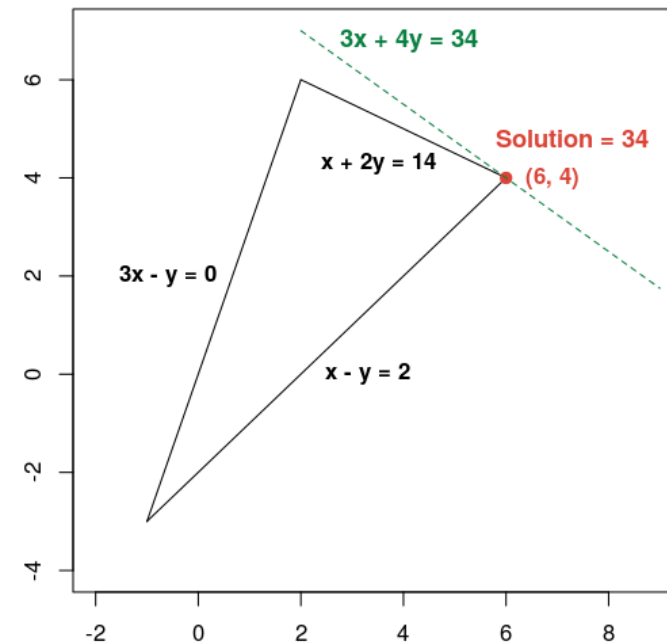
솔버 실행

```
# print_solution
if status == pywraplp.Solver.OPTIMAL:
    print("Solution:")
    print(f"Objective value = {solver.Objective().Value():0.1f}")
    print(f"x = {x.solution_value():0.1f}")
    print(f"y = {y.solution_value():0.1f}")
else:
    print("The problem does not have an optimal solution.")
```

결과 출력

```
# 계산 정보
print("\nAdvanced usage:")
print(f"Problem solved in {solver.wall_time():d} milliseconds")
print(f"Problem solved in {solver.iterations():d} iterations")
```

LinearProgrammingExample()



```
In [1]: %runfile 'C:/Users/1007/Dropbox/04_강의/2021/01_python_예제파일/or1-0.py' --wdir
Number of variables = 2
Number of constraints = 3
Solving with Glop solver v9.11.4210
Solution:
Objective value = 34.0
x = 6.0
y = 4.0

Advanced usage:
Problem solved in 1 milliseconds
Problem solved in 2 iterations

In [2]:
```

실습 1

Maximize

$$3x + y$$

s. t.

$$x + 2y \leq 13.0 \text{ (14} \rightarrow \text{13 변경)}$$

$$3x - y \geq 0.0$$

$$x - y \leq 2.0$$

x, y 는 정수

이대로 문제를 풀면 소수점 값이 나옴.

변수 정수형으로 변경 : `solver.NumVar` $\rightarrow$ `solver.IntVar`,

`x = solver.IntVar(0, solver.infinity(), "x")`

```
In [11]: %runfile 'C:/Users/1007/Dropbox/04_강의/2025-1
01_python_예제파일/or1-0.py' --wdir
Number of variables = 2
Number of constraints = 3
Solving with SCIP 9.0.0 [LP solver: Glop 9.11]
Solution:
Objective value = 31.0
x = 5.0
y = 4.0

Advanced usage:
Problem solved in 2 milliseconds
Problem solved in 0 iterations
```

실습 2

- 앞의 문제에서 x 가 짝수여야 한다는 제약을 추가하여 문제를 푸시오.

Maximize $3x + y$

s. t. $x + 2y \leq 13.0$ (14 → 13 변경)

$$3x - y \geq 0.0$$

$$x - y \leq 2.0$$

제약식 추가

x, y 는 정수이고 x 는 짝수

실습 3

- 다음 정수계획 문제를 배열 데이터 타입으로 정의한 후 최적해를 구하시오.

Maximize $7x_1 + 8x_2 + 2x_3 + 9x_4 + 6x_5$

subject to

$$5x_1 + 7x_2 + 9x_3 + 2x_4 + x_5 \leq 250$$

$$18x_1 + 4x_2 - 9x_3 + 10x_4 + 12x_5 \leq 285$$

$$4x_1 + 7x_2 + 3x_3 + 8x_4 + 5x_5 \leq 211$$

$$5x_1 + 13x_2 + 16x_3 + 3x_4 - 7x_5 \leq 315$$

x_1, x_2, x_3, x_4, x_5 *non - negative integer*

배열을 사용한 모델 정의 – 데이터 정의 (파라미터)

```
from ortools.linear_solver import pywraplp
```

```
data = {}
```

```
data["constraint_coeffs"] = [
```

```
    [5, 7, 9, 2, 1],
```

```
    [18, 4, -9, 10, 12],
```

```
    [4, 7, 3, 8, 5],
```

```
    [5, 13, 16, 3, -7],
```

```
]
```

```
data["bounds"] = [250, 285, 211, 315]
```

```
data["obj_coeffs"] = [7, 8, 2, 9, 6]
```

```
data["num_vars"] = 5
```

```
data["num_constraints"] = 4
```

배열을 사용한 모델 정의 – 솔버 & 변수 선언

```
# Create the mip solver with the SCIP backend.
```

```
solver = pywraplp.Solver.CreateSolver("SCIP")
```

```
infinity = solver.infinity()
```

```
x = {}
```

```
for j in range(data["num_vars"]):
```

```
    x[j] = solver.IntVar(0, infinity, "x[%i]" % j)
```

배열을 사용한 모델 정의 – 제약조건 & 목적함수

제약조건

```
for i in range(data['num_constraints']):
```

```
    constraint_expr = [data['constraint_coeffs'][i][j] * x[j] for j in range(data['num_vars'])]
```

```
    solver.Add(sum(constraint_expr) <= data['bounds'][i])
```

목적함수

```
objective = solver.Objective()
```

```
obj_expr = [data['obj_coeffs'][j] * x[j] for j in range(data['num_vars'])]
```

```
solver.Maximize(solver.Sum(obj_expr))
```

배열을 사용한 모델 정의 – 결과 출력

문제 풀이 시작

```
print(f"Solving with {solver.SolverVersion()}")  
status = solver.Solve()
```

결과 출력

```
if status == pywraplp.Solver.OPTIMAL:  
    print("Objective value =", solver.Objective().Value())  
    for j in range(data["num_vars"]):  
        print(x[j].name(), " = ", x[j].solution_value())  
else:  
    print("The problem does not have an optimal solution.")
```

Classnet Upload

- 실습 1, 2, 3을 하나의 파일로 만들어 클래스 넷에 업로드 할 것